Lightweight 3×3 Depthwise Acceleration with Sliding-Window Reuse in a RISC-V Custom Function Unit for INT8 MobileNetV2

Kasunya Himani
Dept. of Comp. Sci. and Engineering
University of Moratuwa
Colombo, Sri Lanka
kasunya.22@cse.mrt.ac.lk

Mindi Rebeka
Dept. of Comp. Sci. and Engineering
University of Moratuwa
Colombo, Sri Lanka
mindi.22@cse.mrt.ac.lk

Sasmitha Jayasinghe
Dept. of Comp. Sci. and Engineering
University of Moratuwa
Colombo, Sri Lanka
sasmitha.22@cse.mrt.ac.lk

Stephan Balasuriya
Dept. of Comp. Sci. and Engineering
University of Moratuwa
Colombo, Sri Lanka
stephan.22@cse.mrt.ac.lk

Chathuranga Hettiarachchi
Dept. of Comp. Sci. and Engineering
University of Moratuwa
Colombo, Sri Lanka
chathuranga@cse.mrt.ac.lk

***Abstract*—Depthwise convolution remains a significant computational cost in MobileNetV2 inference on resource-constrained RISC-V processors, even when 1×1 pointwise convolutions are hardware-accelerated. This paper presents a dedicated 3×3 depthwise-convolution custom function unit (CFU) integrated into CFU Playground alongside the existing 1×1 datapath. The design combines a pipelined INT8 multiply-accumulate (MAC) datapath with sliding-window caching to reuse overlapping input values and reduce redundant CFU `LOAD` instructions. Evaluated using TensorFlow Lite Micro on a VexRiscv-based LiteX SoC running on a Xilinx Nexys A7-100T at 75 MHz, the depthwise convolution CFU reduces end-to-end inference cycles by 25.8%, achieving a 1.35× speedup over the existing CFU configuration. Sliding-window caching provides a further 6.85% cycle reduction, yielding a 30.8% overall reduction and a 1.45× speedup. It also reduces CFU `LOAD` instructions by 71.3%. The design introduces modest LUT and FF overhead without additional BRAM or DSP usage, demonstrating a lightweight approach to depthwise-convolution acceleration on resource-constrained FPGAs.**

***Index Terms*—MobileNetV2, TinyML, RISC-V, custom function unit, depthwise convolution, FPGA, sliding-window data reuse**

I. Introduction

Embedded convolutional neural network (CNN) inference is expected to fit within tight memory, latency, and energy budgets [1]. MobileNetV2 [2] was designed for such settings by replacing standard convolutions with lightweight pointwise (1×1) and depthwise (3×3) convolutions. Even so, inference still introduces considerable convolutional computation, which can be slow on a small soft processor.

A practical approach is to extend the processor itself. Since RISC-V permits application-specific custom instructions, a custom function unit (CFU) can execute selected compute-intensive operators while an embedded runtime such as TensorFlow Lite Micro (TFLM) [3] executes the remaining operators in software. CFU Playground [4] provides an open-source platform for developing and evaluating custom instructions for TinyML acceleration on FPGAs. One of its existing examples accelerates 1×1 pointwise convolution in MobileNetV2.

Although hardware acceleration of MobileNetV2 has been explored, dedicated 3×3 depthwise-convolution acceleration within the CFU Playground execution path remains underexplored. This work addresses this gap by integrating a dedicated 3×3 depthwise-convolution CFU alongside the existing 1×1 CFU [4] and introducing sliding-window caching to reduce redundant input transfers.

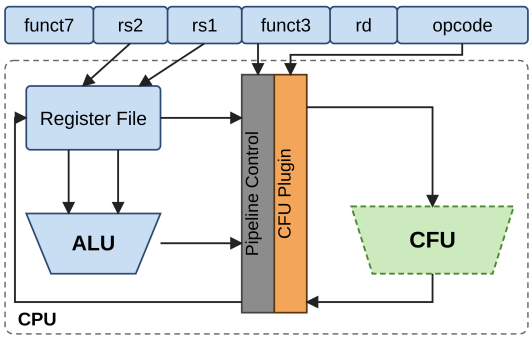


Fig. 1. Overall CPU-CFU architecture

The main contributions of this work are:

- A lightweight 3×3 depthwise convolution CFU integrated alongside the existing 1×1 in the RISC-V/TFLM MobileNetV2 execution path.
- A sliding-window data-reuse mechanism for eligible stride-1 depthwise operations, reducing redundant input transfers to the CFU.
- An end-to-end evaluation of INT8 MobileNetV2 inference covering performance, data movement, FPGA re-

source utilization and timing, and functional correctness.

II. Background and Related Work

A. MobileNetV2 and Depthwise Convolution

Each MobileNetV2 inverted residual block includes a 1×1 expansion, a 3×3 depthwise convolution, and a 1×1 projection [2] with each layer having different computational characteristics. The 1×1 convolution performs a matrix–vector product at each pixel, providing substantial reuse of weights and inputs across output channels and favoring parallel MAC units. Depthwise convolution instead applies a separate 3×3 filter to each channel [5]. Although it requires fewer operations, reuse across channels is limited; adjacent spatial windows therefore provide the main opportunity for input reuse.

B. TFLM and RISC-V CFUs

TFLM executes quantized models through operator-specific kernels [3]. CFU Playground combines TFLM with a LiteX SoC based on the VexRiscv soft processor, allowing selected kernels to invoke custom RISC-V instructions. A CFU can retain internal state, allowing larger operations to be decomposed into short instruction sequences while the remaining operators execute in software. The MobileNetV2 implementation considered here accelerates 1×1 convolution using four-way INT8 MACs and hardware requantization [4].

C. Related Accelerator Approaches

FPGA accelerators span a range of hardware–flexibility trade-offs. RISC-V/CFU designs add hardware for selected operators, including sparse, pruned DNNs [5], while other designs optimize DSP and LUT arithmetic [5], [6]. Yildirim and Ozturk [7] fuse the convolutions of each depthwise-separable block into a single CFU pipeline. This work instead extends the existing tightly coupled CFU with a 3×3 depthwise datapath alongside the 1×1 datapath, keeping operator execution under processor control.

D. Research Gap and Positioning

Existing approaches to depthwise-separable convolution acceleration include specialized fused pipelines that combine multiple convolutional stages in dedicated hardware, potentially at substantial FPGA resource cost [7]. In contrast, the existing CFU Playground MobileNetV2 path accelerates 1×1 pointwise convolution, leaving 3×3 depthwise convolution outside the accelerated path [4]. Extending the existing tightly coupled CFU with a 3×3 depthwise datapath alongside the 1×1 datapath offers a lightweight alternative to fully fused architectures, preserving processor-controlled execution while expanding hardware coverage. This work investigates this approach using a pipelined depthwise datapath and sliding-window reuse to reduce redundant input transfers.

III. Proposed System and CFU Architecture

A. Overall CPU–CFU Architecture

The proposed design extends the existing CFU Playground MobileNetV2 execution path rather than replacing it with a standalone accelerator [4]. TFLM executes the quantized model on the VexRiscv processor, while the CFU is invoked through custom RISC-V instructions.

The single CFU retains the existing 1×1 pointwise convolution datapath [4] and adds a dedicated 3×3 depthwise convolution datapath. Eligible 1×1 expansion and projection operations use the existing datapath, while eligible 3×3 depthwise operations use the new datapath. Unsupported operators remain in software.

For depthwise convolution, the CFU performs the nine-term input-weight accumulation for one output position and channel. The processor handles tensor traversal, addressing, bias addition, requantization, clamping, and output storage operations.

B. 3×3 Depthwise Convolution CFU

For each output position, 3×3 depthwise convolution multiplies and accumulates nine input–weight products per channel. Each of the M input channels is processed independently using its own filter, requiring $9M$ multiply-accumulate (MAC) operations per output spatial position.

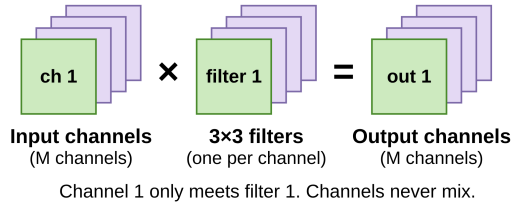


Fig. 2. Every channel is filtered by its own filter

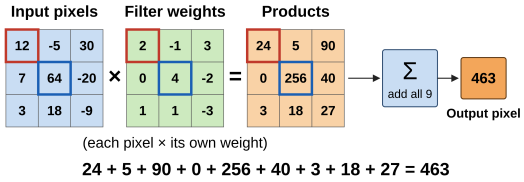


Fig. 3. An example of 3×3 depthwise operation in MobileNetV2

The proposed CFU stores the input window and corresponding weights in local registers, while layer-specific quantization offsets are configured once. The instruction interface consists of `CONFIGURE`, `LOAD`, `RUN`, `GET_RESULT`, and `SHIFT_RIGHT`, summarized in Table I.

TABLE I
3×3 Depthwise-Convolution CFU Instruction Interface.

Instruction	Function	RTL cycles
`CONFIGURE`	Set offsets/reset state	1
`LOAD`	Load input/weight data	1 per instruction
`RUN`	Nine-term MAC	3 or 4*
`GET_RESULT`	Read 32-bit result	1
`SHIFT_RIGHT`	Shift window/insert column	1

* 3 cycles after `LOAD`; 4 cycles after `SHIFT_RIGHT`.

C. Pipelined MAC Architecture

An initial implementation reduced all nine products through a long combinational addition tree, creating a timing-critical path. The final design divides the accumulation into three registered stages. The first stage forms five partial sums, the second reduces these to three values, and the third produces the final result as shown in Fig. 4.

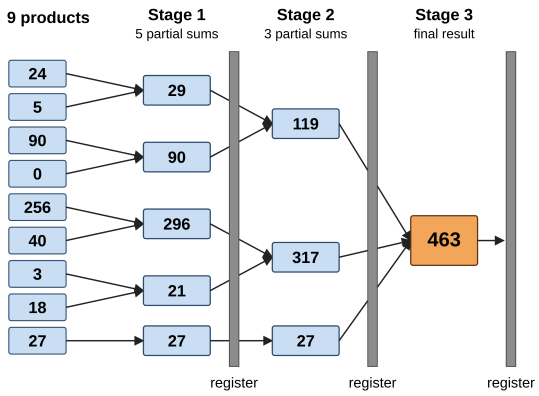


Fig. 4. Pipelined MAC architecture

Registers between stages shorten the combinational path while preserving the nine-term MAC functionality. The same pipeline is used for both newly loaded and reused windows.

D. Sliding Window Data Reuse

The main data-reuse optimization targets consecutive stride-1 depthwise-convolution windows. As shown in Fig. 5, two neighboring 3×3 windows share six of their nine input values. Instead of reloading the complete window for each output position, the stored window is shifted horizontally and only the three newly required input values are supplied.

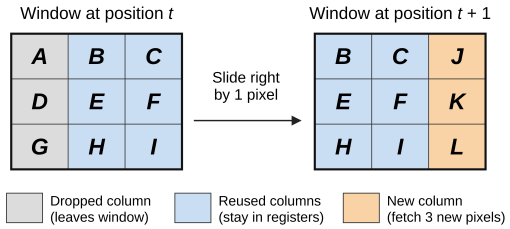


Fig. 5. Sliding window data reuse in 3x3 window

The first window of a row is loaded using

$$\texttt{LOAD} \times 3 \rightarrow \texttt{RUN} \rightarrow \texttt{GET_RESULT},$$

whereas subsequent eligible windows use

$$\texttt{SHIFT_RIGHT} \rightarrow \texttt{RUN} \rightarrow \texttt{GET_RESULT}.$$

Here, `SHIFT_RIGHT` updates the stored window by discarding the leftmost column and inserting the new column; `RUN` then performs the nine-term MAC. The mechanism applies to stride-1 windows within the valid input region. Stride-2 and boundary windows use the conventional loading path.

IV. Experimental Methodology

A. Workload and Platform

All configurations use the same INT8 MobileNetV2 a0.35 model, input image, processor configuration, and system clock so that the incremental effect of the proposed hardware can be isolated. The evaluated model is `mobilenetv2_a035_224_int8.tflite` with an input resolution of 224×224×3 [8]. The external input is UINT8, internal computation uses INT8 arithmetic, and the model produces FLOAT32 outputs for 1000 classes.

The system uses a VexRiscv processor within the LiteX/CFU-Playground SoC and targets the Xilinx Nexys A7-100T development board, featuring an Artix-7 XC7A100T FPGA, at 75 MHz. FPGA implementation, resource utilization, and timing analysis are performed using Vivado 2024.1. TFLM provides the inference runtime and operator kernels. Cycle measurements are obtained during inference on the physical FPGA board using the project's controlled runtime measurement procedure.

B. Experimental Configurations

Three configurations are evaluated incrementally using the same model, input, software environment, and measurement procedure: C1 uses the existing 1×1 CFU, C2 adds the proposed 3×3 depthwise CFU, and C3 additionally enables sliding-window reuse. Table II summarizes the configurations and their evaluation purpose.

TABLE II
EXPERIMENTAL CONFIGURATIONS.

Config.	Description	Purpose
C1	Existing 1×1 CFU	Baseline
C2	C1 + 3×3 depthwise CFU	Measure 3×3 CFU benefit
C3	C2 + sliding-window reuse	Measure reuse benefit

C. Evaluation Metrics

End-to-end inference cycles are the primary performance metric, while per-operator profiling identifies remaining bottlenecks. CFU instruction counts quantify data-transfer activity and are reported separately from execution cycles. FPGA cost is measured using LUT, FF, BRAM, and DSP utilization. Post-route WNS, TNS, and failing endpoints assess timing closure at the 75-MHz target.

D. Verification Methodology

The 3×3 depthwise convolution CFU is verified using 25 gateware tests covering the MAC operation, instruction interface, configuration state, and sliding-window behavior; all tests pass. Software-level equivalence testing compares the optimized depthwise implementation with the reference across supported stride, padding, offset, bias, and quantization cases. Finally, C1–C3 use the same model and embedded input, with output statistics and top-1 classification compared to confirm functional consistency.

V. Results and Discussion

A. End-to-End Performance

Table III summarizes the measured execution of the three configurations.

TABLE III
END-TO-END PERFORMANCE AND 3×3 CFU INSTRUCTION ACTIVITY.

Metric	C1	C2	C3
Inference cycles	828,991,457	615,396,021	573,258,998
Speedup vs. C1	1.00×	1.35×	1.45×
Cycle reduction vs. C1	–	25.8%	30.8%
Depthwise share of cycles	70.4%	60.1%	55.3%
LOAD instructions	–	2,529,552	726,864
SHIFT_RIGHT instructions	–	0	600,896

Adding the 3×3 depthwise CFU reduces inference cycles by 25.8% (1.35× speedup) relative to C1. Sliding-window reuse provides a further 6.85% reduction over C2, reaching a 30.8% reduction (1.45× speedup) over C1. The depthwise cycle share falls from 70.4% in C1 to 55.3% in C3, indicating the growing contribution of software and other unaccelerated operations.

B. Effect of Sliding-Window Reuse

At the RTL level, a newly loaded window requires 7 CFU cycles, whereas a reused window requires 6, including the additional product-refresh cycle after `SHIFT_RIGHT`. These counts describe RTL-level CFU latency and should not be interpreted as independently verified CPU-visible latency.

Sliding-window reuse reduces `LOAD` instructions from 2,529,552 in C2 to 726,864 in C3, a 71.3% reduction, while introducing 600,896 `SHIFT_RIGHT` instructions. The smaller end-to-end improvement reflects the additional costs of CFU invocation, result retrieval, and processor-side processing.

C. FPGA Resource Utilization and Timing

Table IV summarizes the FPGA implementation results.

TABLE IV
FPGA RESOURCE UTILIZATION AND TIMING RESULTS.

Metric	C1	C2	C3
LUT (total / CFU)	6,187 / 1,511	6,865 / 1,678	8,017 / 2,693
FF (total / CFU)	4,504 / 1,081	4,898 / 1,475	5,080 / 1,657
RAMB36 / RAMB18	15 / 33	15 / 33	15 / 33
DSP	8	8	8
WNS (ns)	+0.962	+0.372	+0.322
Failing endpoints	0	0	0

Adding the 3×3 datapath increases CFU utilization by 167 LUTs and 394 FFs relative to C1, with no additional BRAM or DSP slices. C3 introduces further state and control logic for window reuse, trading additional logic for the 6.85% performance improvement over C2. All configurations meet the 75-MHz target with zero failing endpoints; C3 achieves a post-route WNS of +0.322 ns.

D. Limitations and Future Work

The sliding-window optimization currently supports eligible stride-1 windows; stride-2 and boundary cases use conventional loading. Processor-controlled traversal and post-processing also limit the end-to-end benefit. As a next step toward larger-kernel depthwise convolution, the validated 3×3 datapath can be extended to 5×5 operations. This requires further optimization of the multiplication datapath, operand storage, and data movement under FPGA resource constraints. The resulting accelerator can then be evaluated on MobileNetV3 workloads containing both 3×3 and 5×5 depthwise convolutions, with further dataflow and buffering optimizations explored to increase data reuse and reduce processor–accelerator transfers.

VI. Conclusion

This work extended the RISC-V/TFLM MobileNetV2 execution path with a lightweight, pipelined 3×3 depthwise convolution CFU alongside the existing 1×1 CFU. On INT8 MobileNetV2, the depthwise CFU reduced end-to-end inference cycles by 25.8% (1.35×), and sliding-window reuse extended this to 30.8% (1.45×) while cutting CFU `LOAD` instructions by 71.3%. The functionally verified design required no additional BRAM or DSP resources and met the 75-MHz target with zero failing endpoints.

References

[1] C. Banbury, V. J. Reddi, P. Torelli, J. Holleman, N. Jeffries, C. Kiraly, P. Montino, D. Kanter, S. Ahmed, D. Pau *et al.*, "Mlperf tiny benchmark," *arXiv preprint arXiv:2106.07597*, 2021.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *2018 IEEE/CVF conference on computer vision and pattern recognition*. Ieee, 2018, pp. 4510–4520.

[3] R. David, J. Duke, A. Jain, V. Janapa Reddi, N. Jeffries, J. Li, N. Kreeger, I. Nappier, M. Natraj, T. Wang *et al.*, "Tensorflow lite micro: Embedded machine learning for tinyml systems," *Proceedings of machine learning and systems*, vol. 3, pp. 800–811, 2021.

[4] S. Prakash, T. Callahan, J. Bushagour, C. Banbury, A. V. Green, P. Warden, T. Ansell, and V. J. Reddi, "Cfu playground: Full-stack open-source framework for tiny machine learning (tinyml) acceleration on fpgas," in *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2023, pp. 157–167.

[5] M. Sabih, A. Karim, J. Wittmann, F. Hannig, and J. Teich, "Hardware/software co-design of risc-v extensions for accelerating sparse dnns on fpgas," in *2024 International Conference on Field Programmable Technology (ICFPT)*. IEEE, 2024, pp. 01–09.

[6] M. Véstias, R. P. Duarte, J. T. De Sousa, and H. Neto, "Parallel dot-products for deep learning on fpga," in *2017 27th international conference on field programmable logic and applications (FPL)*. IEEE, 2017, pp. 1–4.

[7] M. Yildirim and O. Ozturk, "Risc-v based tinyml accelerator for depthwise separable convolutions in edge ai," *arXiv preprint arXiv:2511.21232*, 2025.

[8] STMicroelectronics, "Stm32 ai model zoo," GitHub repository, accessed: Oct. 8, 2026. [Online]. Available: https://github.com/STMicroelectronics/stm32ai-modelzoo